Inventory Management System Installation

This document describes how to install and run the Inventory Management System (IMS) Flask server with Python and SQLite.

Recommended Server

The recommended server operating system is **Ubuntu Server 24.04 LTS**.

Minimum development or VM requirements:

- 1 CPU core
- 1 GB RAM
- 10 GB available disk space
- SSH access for remote administration

Required Software

The server requires:

- Python 3.12 or newer
- Python virtual environment support
- Git
- Flask
- SQLite 3

Ubuntu Installation

Install the system packages:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-venv python3-pip git sqlite3
```

This installs Python, the virtual environment tools, pip, Git, and SQLite.

After that, verify the installation:

Verify the installation:

```
python3 --version
sqlite3 --version
```

Download the Project

Clone the project and enter its directory:

```
git clone https://github.com/mhbuys/Inventory-Management-System.git
cd Inventory-Management-System
```

Create the Python Environment

Create a project-local virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
```

Install the Python dependencies:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The virtual environment should be activated whenever the server is run manually.

Initialize SQLite

Create the application data directory and make sure the current user can write to it:

```
mkdir -p instance
sudo chown -R $USER:$USER instance
chmod -R u+rwX instance
```

Then initialize the database:

```
flask --app app init-db
```

This also creates the default admin account:

```
username: admin
password: admin
```

The SQLite database will be stored at:

```
instance/inventory.db
```

The database will contain the IMS data needed for Products and Inventory Events. Product records will support the documented fields: product ID, product name, category, quantity in stock, reorder threshold, reorder quantity, and product status.

Test the Database Connection

Confirm the app can reach SQLite before starting the main server:

```
flask --app app test-db-connection
```

Expected output:

```
connected
```

If it fails, the command will print:

```
not connected
```

Run the Flask Server

Start Flask so it can be reached from your server:

```
flask --app app run --host 0.0.0.0 --port 5000
```

Replace `(0.0.0.0)` with the actual IP address of the machine running the app.

Open the server from another computer using:

```
http://(your server IP):5000
```

For local-only testing, use `127.0.0.1` instead of `(your server IP)`.

Open the Firewall Port

If Ubuntu's firewall is enabled, allow Flask's development port for testing:

```
sudo ufw allow 5000/tcp
sudo ufw status
```

Production Direction

The development server is suitable for local development and initial VM testing. For production, the planned deployment is:

1. Flask application
2. A systemd service to start the IMS automatically

Those services will be added after the initial Flask and SQLite application is working.

Windows Development

The server can also be developed on Windows with Python 3.12 or newer:

```
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
flask --app app init-db
flask --app app run
```

Ubuntu Server remains the recommended VM operating system because the deployment commands and systemd setup are simpler there.

Current Database Decision

The original requirements document references MySQL, but the current implementation direction is SQLite. SQLite is appropriate for the initial single-server IMS because it is simple to install, requires no separate service, and keeps the database in the project instance directory.

The database access layer should remain isolated behind Python functions so the IMS can migrate to MySQL later without changing the Flask routes or inventory rules.